GENAI CAPSTONE PROJECT

Credit Risk Prediction

Milestone 1 — ML Pipeline

Milestone 2 — Agentic AI System

32,576	91.45%	0.891	3
Applicant Records	Best Accuracy	ROC-AUC Score	Agent Nodes

Logistic Regression · Decision Tree · LangGraph · RAG + LLaMA 3.1 · ChromaDB

Table of Contents

1 Executive Summary **3**

2 Dataset Overview **4**

2.1 Raw Dataset 4

2.2 Descriptive Statistics 5

3 Exploratory Data Analysis **5**

3.1 Target Variable Distribution 5

3.2 Categorical Feature Distributions 6

4 Data Preprocessing **7**

4.1 Data Quality 7

4.2 Feature–Target Separation 7

4.3 Label Encoding 7

5 Train / Test Split & Feature Scaling **8**

5.1 Stratified 80/20 Split 8

5.2 StandardScaler 8

6 Model Training **9**

6.1 Model 1 — Logistic Regression (Baseline) 9

6.2 Model 2 — Decision Tree Classifier 10

7 Model Evaluation **10**

7.1 Head-to-Head Summary 11

7.2 Classification Reports 11

7.2.1 Logistic Regression 11

7.2.2 Decision Tree 12

7.3 Confusion Matrices 12

7.4 ROC Curves 13

8 Feature Importance & Overfitting **14**

8.1 Feature Importance (Decision Tree) 14

8.2 Overfitting Check 15

9 Model Deployment — Milestone 1 **15**

10 Agentic System Overview **17**

10.1 What Makes This System “Agentic”? 17

10.2 System Architecture 18

10.3 Technology Stack 18

11 Agent Node 1 — ML Prediction Tool **18**

11.1 Input Preprocessing 18

11.2 Rule-Based Safety Overrides 19

12 Agent Node 2 — RAG Knowledge Retrieval **20**
12.1 Knowledge Base Construction 20
12.2 RAG Tool Logic 20
13 Agent Node 3 — LLM Reasoning Tool **21**
13.1 LLM Configuration 21
13.2 Prompt Design 21
14 LangGraph Workflow Assembly **22**
15 Agent Testing & Validation **22**
15.1 Sample Prediction Output 23
15.2 Deterministic Boundary Tests 23
15.3 Credit Score Sensitivity Analysis 23
15.4 Randomised Stress Test 24
16 Key Takeaways & Future Improvements **24**
16.1 Milestone 1 — Design Decisions 24
16.2 Milestone 2 — Design Decisions 25
16.3 Final Results 25
16.4 Future Improvements 25
Conclusion **27**

1. Executive Summary

This report covers the complete **GenAI Capstone Project** across two milestones. **Milestone 1** delivers a production-ready binary classification pipeline that predicts loan default risk from 32,576 applicant records. **Milestone 2** wraps that ML model inside a three-node **Agentic AI system** powered by LangGraph, Retrieval-Augmented Generation (RAG), and the LLaMA 3.1 8B language model — producing explainable, rule-grounded credit decisions in natural language.

Dataset	32,576 applicants × 12 features (Kaggle: laavanya/credit-risk-prediction-dataset)
Encoding	Label Encoding (LabelEncoder) — 10-feature vector
Target	`loan_status`: 0 = Good Loan, 1 = Default
ML Winner	Decision Tree (max_depth=10) — 91.45% accuracy, ROC-AUC 0.8908
Agent	LangGraph: Node 1 (ML) → Node 2 (RAG/ChromaDB) → Node 3 (LLaMA 3.1 8B via Groq)
Output	Structured APPROVE/REJECT decision + natural-language justification

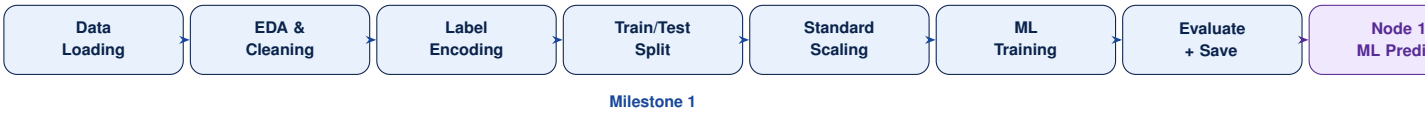


Figure 1 End-to-end pipeline — Milestone 1 (blue) feeds the Agentic AI system of Milestone 2 (violet)

Milestone 1 — Machine Learning Pipeline

2. Dataset Overview

2.1 Raw Dataset

The dataset `cleaned_credit_risk.csv` contains **32,576 rows** and **12 columns** with zero missing values. `loan_grade` is dropped before modelling to prevent data leakage; `loan_status` is the binary target.

Table 1 Complete Feature Dictionary

#	Feature	Dtype	Description	Role
1	`person_age`	int64	Applicant age (years)	Input
2	`person_income($)`	int64	Annual income (USD)	Input
3	`person_home_ownership`	object	Home ownership status	Input → LE
4	`person_emp_length`	float64	Employment duration (years)	Input
5	`loan_intent`	object	Purpose of loan	Input → LE
6	`loan_grade`	object	Credit grade A–G	*Dropped*
7	`loan_amnt($)`	int64	Loan amount (USD)	Input
8	`loan_int_rate`	float64	Loan interest rate (%)	Input
9	`loan_status`	int64	**Target**: 0=Good, 1=Default	Target
10	`loan_percent_income`	float64	Loan-to-income ratio	Input
11	`cb_person_default_on_fi`	object	Prior default on record	Input → LE
12	`cb_person_cred_hist_len`	int64	Credit history length (years)	Input

2.2 Descriptive Statistics

Table 2 Key Numerical Feature Statistics (Training Set, $n = 26{,}060$)

Feature	Mean	Std	Min	25%	Median	Max
`person_age`	27.68	6.15	20	23	26	94
`person_income($)`	65,833	52,076	4,000	38,568	55,000	2.04M
`person_emp_length`	4.75	3.99	0.0	2.0	4.0	60.0
`loan_amnt($)`	9,393	5,798	500	5,000	8,000	23,000
`loan_int_rate`	11.00	3.07	5.42	8.49	10.99	22.48
`loan_percent_income`	0.170	0.107	0.000	0.090	0.150	0.830

3. Exploratory Data Analysis

3.1 Target Variable Distribution

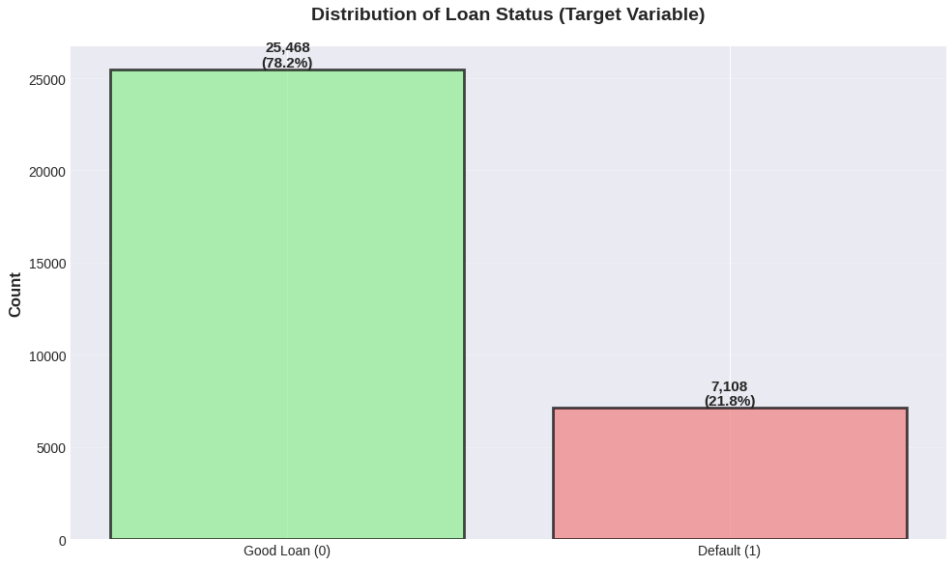
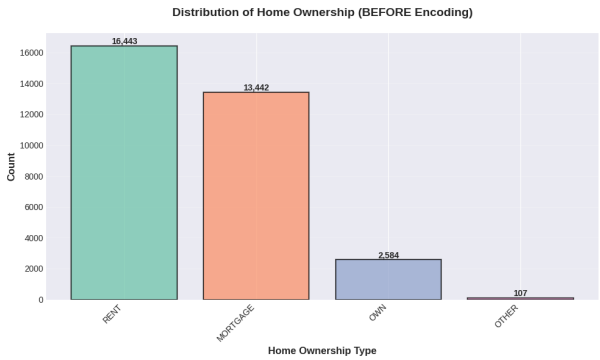


Figure 2 Loan status distribution — 78.2% good loans vs 21.8% defaults across 32,576 records

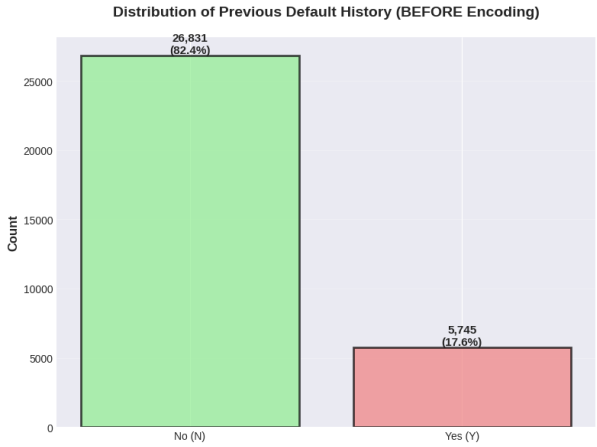
▲ Class Imbalance

The Good : Default ratio is **3.58 : 1** (25,468 vs 7,108). This imbalance is not addressed by threshold tuning in this version — the default 0.50 threshold is used with both models. Future work should consider SMOTE, class-weight balancing, or threshold optimisation.

3.2 Categorical Feature Distributions



(a) Home ownership type



(b) Prior credit default on file

Figure 3 Categorical feature distributions before Label Encoding

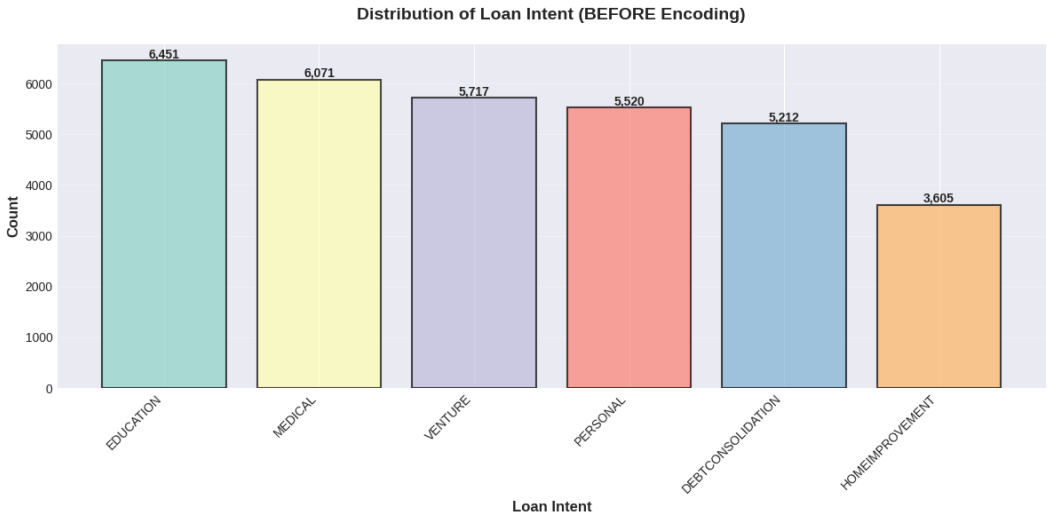


Figure 4 Loan intent — Education and Medical are the largest groups; all six intents well-represented

Table 3 Categorical Feature Summary

Feature	Category	Count	Share
home_ownership	RENT	16,443	50.5%
	MORTGAGE	13,442	41.3%
	OWN	2,584	7.9%
	OTHER	107	0.3%
loan_intent	EDUCATION	6,451	19.8%
	MEDICAL	6,071	18.6%
	VENTURE	5,717	17.5%
	PERSONAL	5,520	16.9%
	DEBTCONSOLIDATION	5,212	16.0%
	HOMEIMPROVEMENT	3,605	11.1%
default_on_file	No (N)	26,831	82.4%
	Yes (Y)	5,745	17.6%

4. Data Preprocessing

4.1 Data Quality

All 12 columns contain **zero missing values** across every record. The dataset was pre-cleaned (outlier handling, age filtering to 18–100, employment length capped at 60 years, column renaming for clarity).

4.2 Feature–Target Separation

`loan_grade` is removed before any encoding step to prevent target leakage — it is derived directly from creditworthiness outcomes. The feature matrix before encoding is **10 columns** (7 numerical + 3 categorical).

4.3 Label Encoding

Three categorical columns are encoded with scikit-learn's `LabelEncoder`. The encoders are serialised inside the model package for consistent inference.

Listing 1 Label Encoding of Categorical Variables

```
cat_cols = ["person_home_ownership", "loan_intent", "cb_person_default_on_file"
    ]
encoders = {}

for col in cat_cols:
    le = LabelEncoder()
    X[col] = le.fit_transform(X[col])
```

```
    encoders[col] = le    # stored for deployment
```

Table 4 Label Encoding Mappings

Feature	Mapping
`person_home_ownership`	MORTGAGE→0 OTHER→1 OWN→2 RENT→3
`loan_intent`	DEBTCONSOLIDATION→0 EDUCATION→1 HOMEIMPROVEMENT→2 MEDICAL→3 PERSONAL→4 VENTURE→5
`cb_person_default_on_file`	N→0 Y→1

5. Train / Test Split & Feature Scaling

5.1 Stratified 80/20 Split

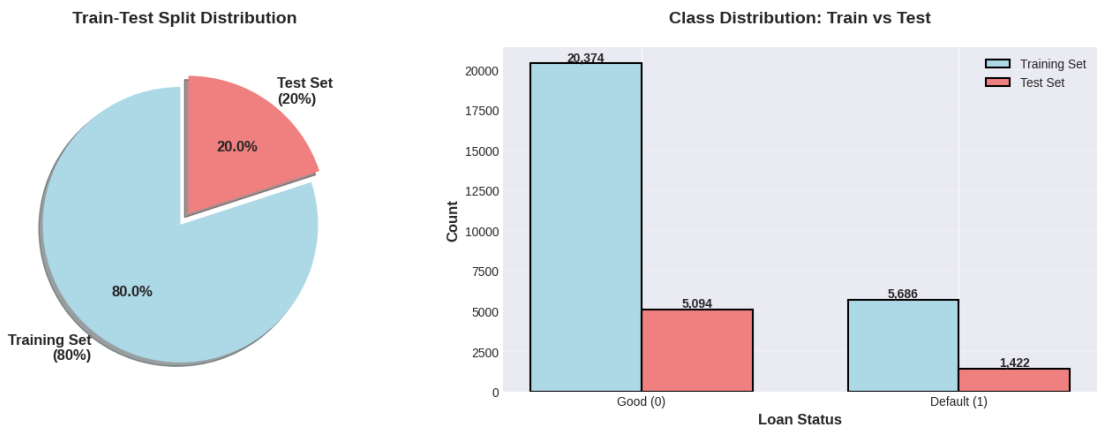


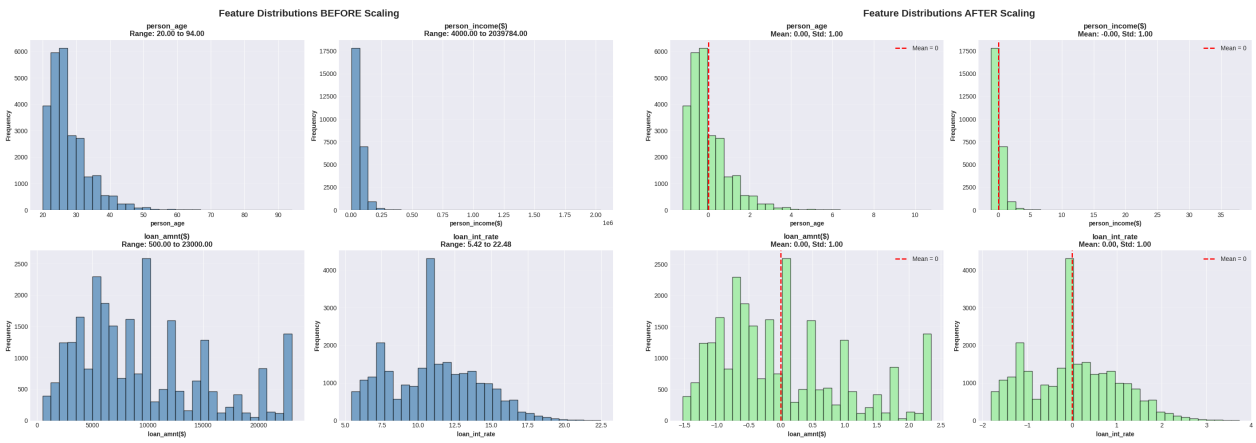
Figure 5 Train–test split visualisation — class proportions preserved identically in both subsets

Table 5 Split Summary with Class Stratification Verification

Subset	Total	Share	Good (0)	Default (1)	Shape
Full dataset	32,576	100%	25,468 (78.18%)	7,108 (21.82%)	(32576, 10)
Training set	26,060	80%	20,374 (78.18%)	5,686 (21.82%)	(26060, 10)
Test set	6,516	20%	5,094 (78.18%)	1,422 (21.82%)	(6516, 10)

5.2 StandardScaler

The scaler is **fitted on training data only** to prevent leakage: $z = (x - \mu_{\text{train}})/\sigma_{\text{train}}$.



(a) Distributions **before** scaling **(b)** Distributions **after** scaling

Figure 6 StandardScaler normalises all features to mean ≈ 0, std ≈ 1

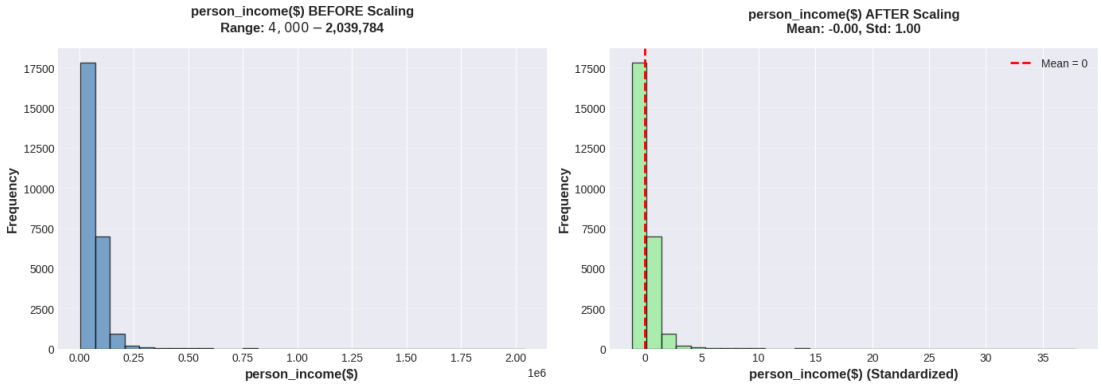


Figure 7 Person income: before vs. after standardisation — shape preserved, scale unified across all features

6. Model Training

6.1 Model 1 — Logistic Regression (Baseline)

Listing 2 Logistic Regression — Training

```
lr_model = LogisticRegression(max_iter=1000, random_state=42)
lr_model.fit(X_train_scaled, y_train)
# Result: 10 features | Converged in 15 iterations | Solver: lbfgs
```

Table 6 Logistic Regression Configuration

Parameter	Value
Solver	lbfgs (L-BFGS-B, default)
Regularisation	L2, $C = 1.0$
`max_iter`	1,000
Actual iterations	15 (early convergence)
Input features	10
Threshold	0.50 (default)

6.2 Model 2 — Decision Tree Classifier

Listing 3 Decision Tree — Training

```
dt_model = DecisionTreeClassifier(max_depth=10, random_state=42)
dt_model.fit(X_train_scaled, y_train)
# Result: depth=10 | 264 leaf nodes | 10 features
```

Table 7 Decision Tree Configuration

Parameter	Value
Criterion	Gini impurity
`max_depth`	10
Actual depth	10
Leaf nodes	264
Input features	10
Threshold	0.50 (default)

7. Model Evaluation

7.1 Head-to-Head Summary

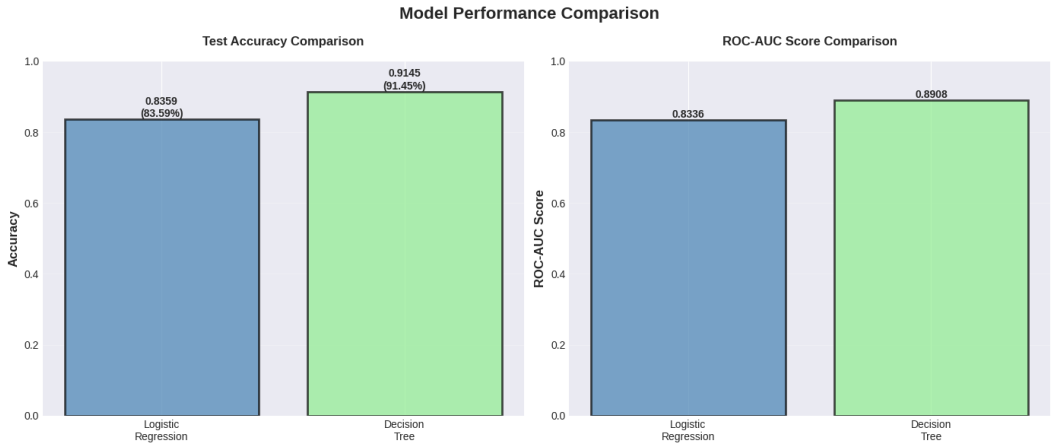


Figure 8 Model performance dashboard — Decision Tree wins convincingly on both accuracy and ROC-AUC

Table 8 Overall Performance Comparison

Metric	Logistic Regression	Decision Tree	Winner
Test Accuracy	83.59%	**91.45%**	Decision Tree
ROC-AUC Score	0.8336	**0.8908**	Decision Tree
Training Accuracy	—	92.88%	—
Overfitting Gap	—	**1.43%**	Excellent

7.2 Classification Reports

7.2.1 Logistic Regression

Table 9 Logistic Regression — Classification Report (Test Set, $n = 6{,}516$)

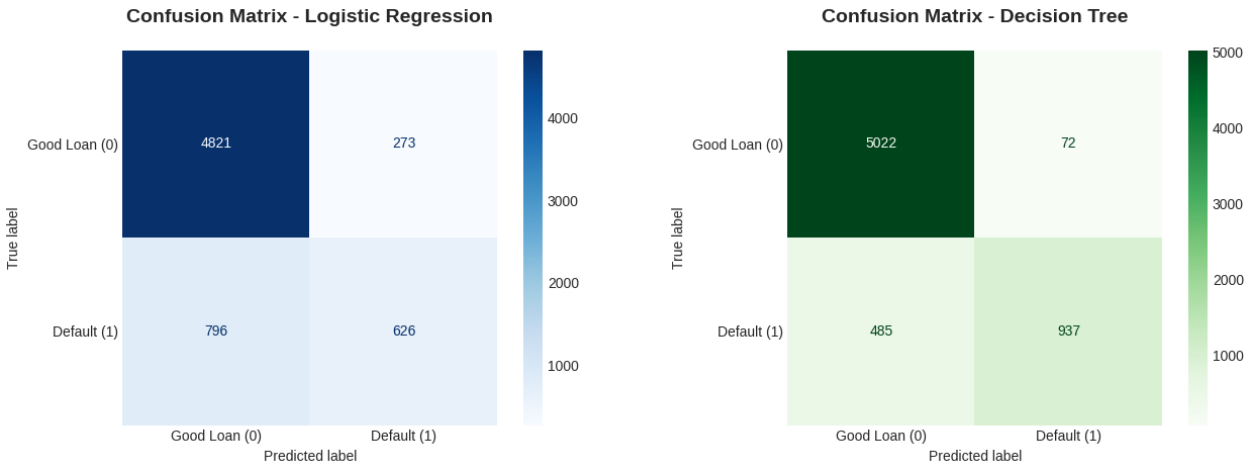
Class	Precision	Recall	F1-Score	Support
Good Loan (0)	0.86	0.95	0.90	5,094
Default (1)	0.70	0.44	0.54	1,422
Macro avg	0.78	0.69	0.72	6,516
Weighted avg	0.82	0.84	0.82	6,516
Accuracy			**0.8359 (83.59%)**	**6,516**

7.2.2 Decision Tree

Table 10 Decision Tree — Classification Report (Test Set, $n = 6{,}516$)

Class	Precision	Recall	F1-Score	Support
Good Loan (0)	0.91	0.99	0.95	5,094
Default (1)	0.93	0.66	0.77	1,422
Macro avg	0.92	0.82	0.86	6,516
Weighted avg	0.92	0.91	0.91	6,516
Accuracy			**0.9145 (91.45%)**	**6,516**

7.3 Confusion Matrices



(a) Logistic Regression **(b)** Decision Tree

Figure 9 Confusion matrices — Decision Tree reduces False Negatives by 311 and False Positives by 201

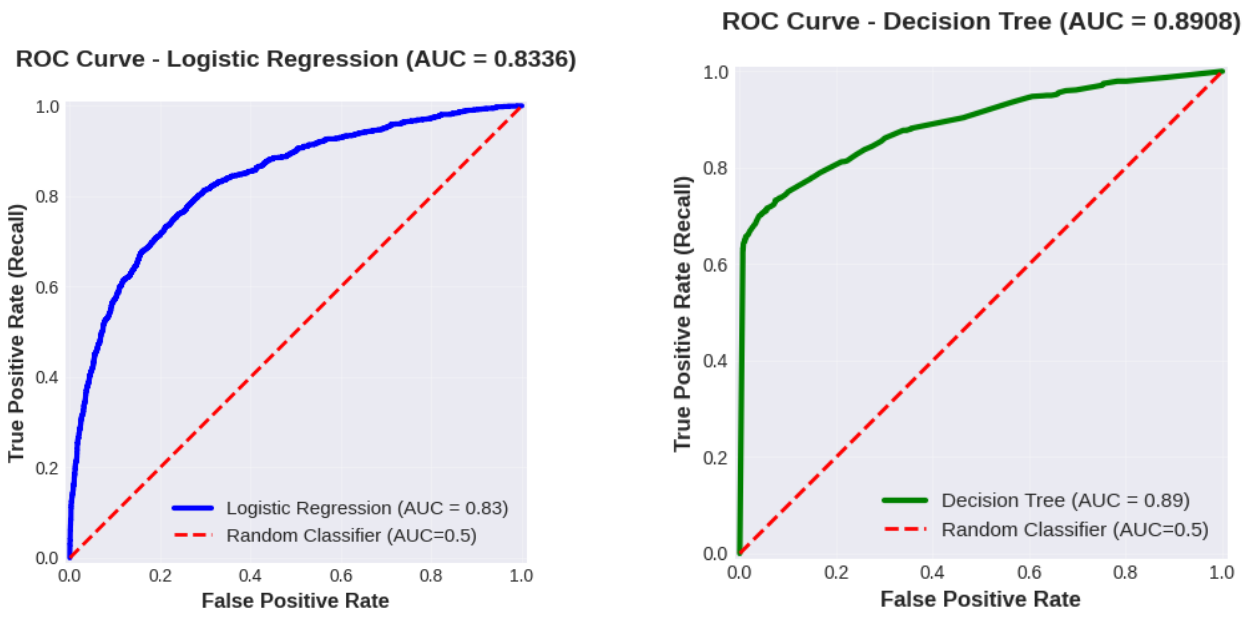
Table 11 Confusion Matrix Breakdown — Business Impact

	Metric	LR	DT	Business Meaning
TN	Correct approvals	4,821	5,022	Revenue preserved
FP	Good loans rejected	273	72	Lost opportunity
FN	Missed defaults	796	485	Financial loss
TP	Defaults caught	626	937	Risk avoided

Business Impact

The Decision Tree catches **311 more defaults** (937 vs 626) and rejects **201 fewer creditworthy customers** (72 vs 273) compared to Logistic Regression. Both improvements directly translate to lower financial loss and better customer retention.

7.4 ROC Curves



(a) Logistic Regression (AUC = 0.8336) **(b)** Decision Tree (AUC = 0.8908)

Figure 10 Individual ROC curves — both models substantially outperform the random baseline (AUC = 0.50)

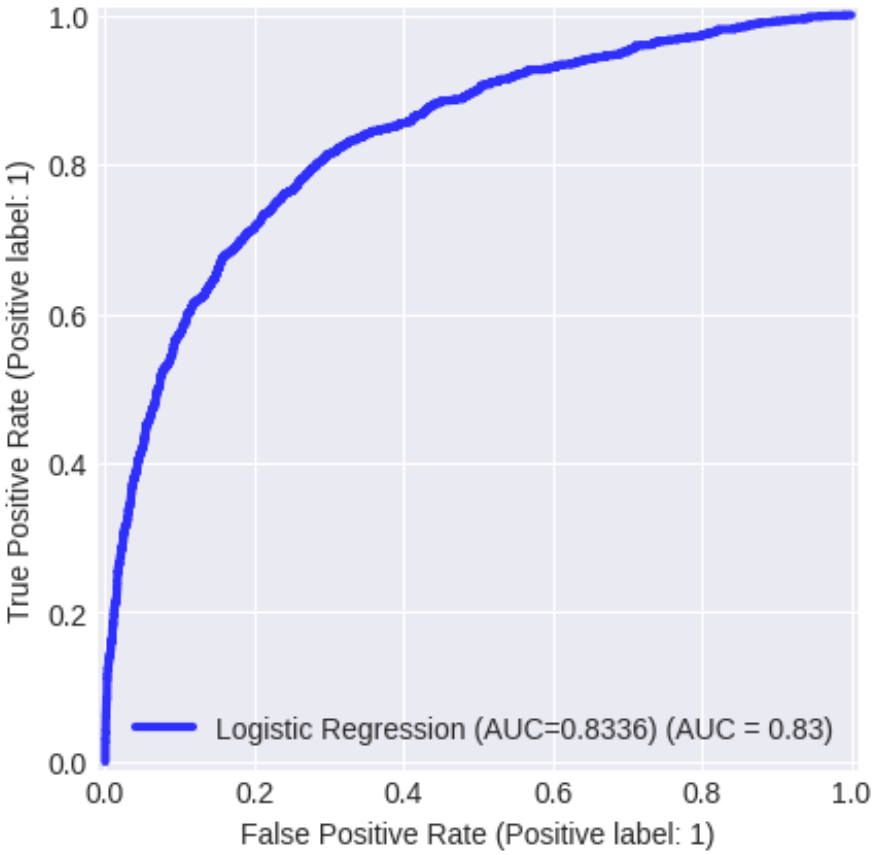


Figure 11 Combined ROC comparison — Decision Tree maintains higher TPR across all operating thresholds

8. Feature Importance & Overfitting

8.1 Feature Importance (Decision Tree)

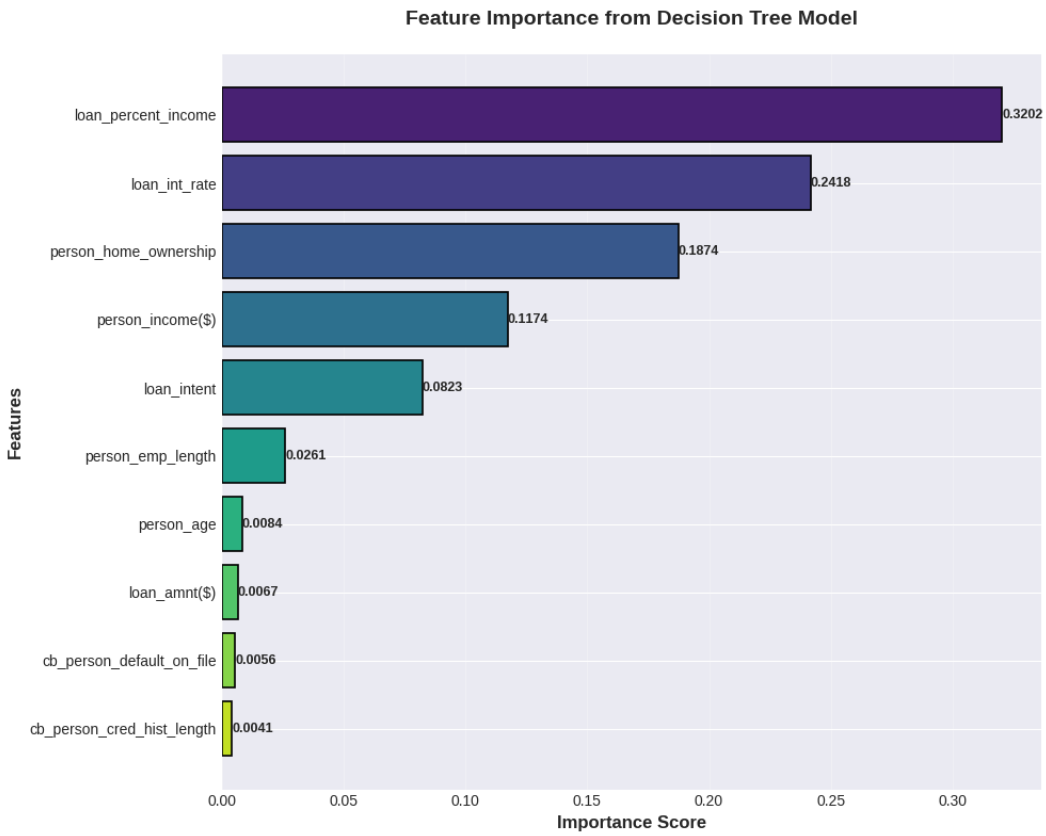


Figure 12 Gini-impurity-based feature importance — top two features drive over 56% of all decisions

Table 12 Feature Importance Rankings

Rank	Feature	Score	Cumul.	Business Meaning
1	`loan_percent_income`	0.3202	32.0%	Debt burden relative to income
2	`loan_int_rate`	0.2418	56.2%	True cost of borrowing
3	`person_home_ownership`	0.1874	74.9%	Asset stability proxy
4	`person_income($)`	0.1174	86.7%	Absolute repayment capacity
5	`loan_intent`	0.0823	94.9%	Loan purpose risk profile
6–10	*Remaining features*	0.0509	100%	Minor contributions

8.2 Overfitting Check

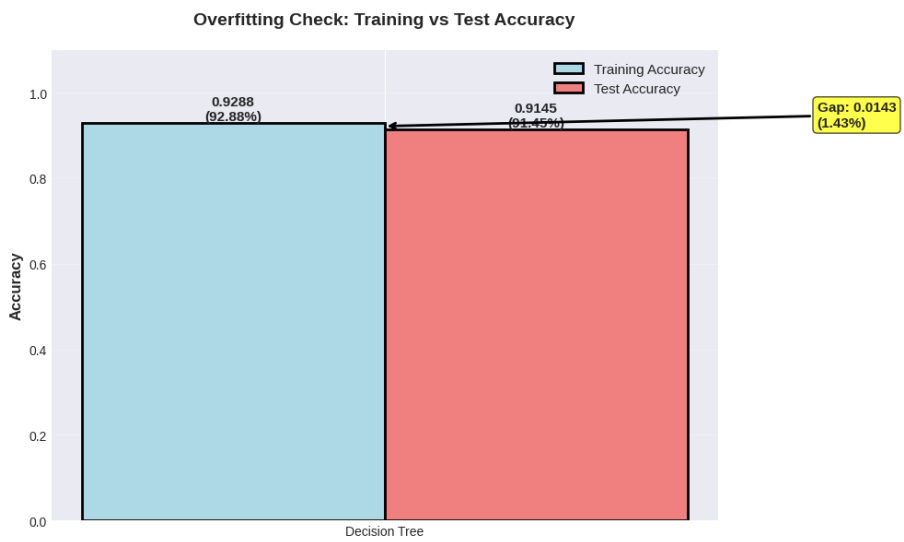


Figure 13 Training vs. test accuracy — 1.43% gap confirms excellent generalisation with max_depth=10

✔ Result

Training accuracy 92.88% vs. test accuracy 91.45% gives an overfitting gap of only **1.43%** — well within the "excellent" threshold of 2%. The `max_depth=10` constraint successfully regularises the tree.

9. Model Deployment — Milestone 1

Listing 4 Building and Saving the Deployment Package

```
model_package = {
    "model":            dt_model,            # DecisionTreeClassifier
    "scaler":           scaler,              # Fitted StandardScaler (10 features
        )
    "encoders":         encoders,            # 3 LabelEncoder instances
    "feature_columns": X.columns.tolist()  # Ordered feature list
}
with open("dt_model.pkl", "wb") as f:
    pickle.dump(model_package, f)
# File: dt_model.pkl | 44,534 bytes (43.49 KB) | Lossless round-trip verified
```

Table 13 Deployment Package Contents

Component	Type	Size	Purpose
`model`	DecisionTreeClassifier	∼43 KB	Core inference engine
`scaler`	StandardScaler	<1 KB	Normalise incoming features
`encoders`	dict[str, LabelEncoder]	<1 KB	Encode 3 categorical fields
`feature_columns`	list (10 items)	Bytes	Enforce correct column order
File	`dt_model.pkl`	**43.49 KB**	Self-contained, portable

The model was verified by loading the `.pkl` and confirming predictions are **bit-for-bit identical** to the original in-memory model. A real-time simulation on 50 randomly generated applicants demonstrated the full preprocess → encode → scale → predict pipeline working end-to-end without errors.

Milestone 2 — Agentic AI System for Credit Risk Assessment

10. Agentic System Overview

Milestone 2 extends the Milestone 1 pipeline by wrapping the trained Decision Tree inside a **three-node LangGraph agent**. Rather than returning a bare binary prediction, the agent produces a **structured, natural-language credit decision** grounded in both the ML output and retrieved lending-domain policy rules — making every decision explainable and auditable.

10.1 What Makes This System "Agentic"?

Table 14 Agentic Properties of the System

Property	Implementation
Tool use	Agent calls three distinct tools: ML model, vector store (ChromaDB), LLM
Context awareness	Retrieved domain rules are injected into the LLM prompt alongside ML output
Structured decisions	LLM is explicitly instructed to follow ML probability — no hallucinated approvals
State propagation	Shared state dict (`ml_output`, `rules`, `analysis`) flows through every node
Rule-based safety	Hard probability floors override the ML for extreme edge cases
Extensibility	New nodes (fraud check, API call) can be inserted without changing existing logic

10.2 System Architecture

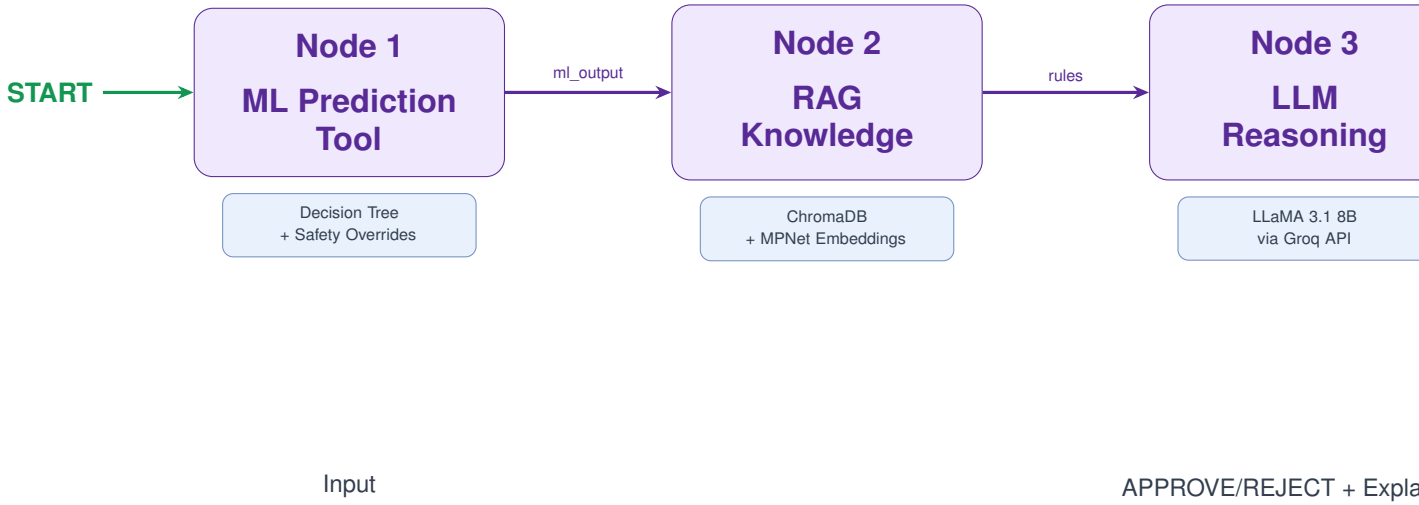


Figure 14 LangGraph three-node pipeline — shared state dict flows left to right through each node

10.3 Technology Stack

Table 15 Milestone 2 Technology Stack

Library	Version	Role	Why
`langgraph`	Latest	Agent orchestration	Graph-based state routing
`langchain-community`	Latest	LLM / retriever wrapper	Chroma + OpenAI interface
`langchain-huggingface`	Latest	Embeddings	HuggingFace model loading
`chromadb`	Latest	Vector store	Local, in-memory RAG
`sentence-transformers`	Latest	Text embedding	all-mpnet-base-v2
`groq`	Latest	LLM provider client	Ultra-low latency inference
`langchain-openai`	Latest	LLM interface	OpenAI-compatible Groq endpoint

11. Agent Node 1 — ML Prediction Tool

★ Node 1: `predict_risk_tool`

Receives raw applicant input, applies preprocessing (defaults + encoding + scaling), calls the Milestone 1 Decision Tree, and applies rule-based safety overrides before writing `ml_output` to shared state.

11.1 Input Preprocessing

The `preprocess_input()` function accepts a simplified input dictionary (e.g., `income`, `credit_score`, `employment_years`) and maps it to the full 10-feature vector using dataset medians as defaults:

Table 16 Feature Defaults Used in `preprocess_input()`

Feature	Default	Source
`person_age`	26	Median from `df.describe()`
`person_income($)`	55,000	Median
`person_home_ownership`	RENT	Mode
`person_emp_length`	4.0	Median
`loan_intent`	EDUCATION	Mode
`loan_amnt($)`	8,000	Median
`loan_int_rate`	10.99	Median
`loan_percent_income`	0.15	Median
`cb_person_default_on_file`	N	Mode
`cb_person_cred_hist_length`	4	Median

11.2 Rule-Based Safety Overrides

The Decision Tree may underestimate risk for extreme inputs not well-represented in the training distribution. Three hard probability floors are applied *after* the model prediction, using `max(prob, floor)` so they only ever raise — never lower — the estimated risk:

Table 17 Safety Override Rules in `predict_risk_tool`

Condition	Min. Probability	Rationale
`income` $\leq 10{,}000$	**0.70**	Extremely low income makes repayment structurally unlikely
`credit_score` < 400	**0.80**	Very short credit history signals high uncertainty
`employment_years` $= 0$	**0.75**	No employment eliminates the primary repayment source

Listing 5 Rule-Based Safety Overrides in Node 1

```
# ML prediction
prob = model.predict_proba(processed)[0][1]

# Safety floors      only raise probability, never lower it
if data["income"] <= 10000:           prob = max(prob, 0.70)
if data["credit_score"] < 400:        prob = max(prob, 0.80)
if data["employment_years"] == 0:     prob = max(prob, 0.75)

state["ml_output"] = {"prediction": int(pred), "probability": float(prob)}
```

12. Agent Node 2 — RAG Knowledge Retrieval

★ Node 2: `rag_tool`

Performs a cosine-similarity search over the embedded lending policy documents in ChromaDB and writes the top-4 retrieved rules into `state["rules"]` for the LLM to ground its explanation.

12.1 Knowledge Base Construction

Five domain-specific lending policy documents are embedded using **sentence-transformers/all-mpnet-base-v2** and stored in a local Chroma vector store:

Table 18 RAG Knowledge Base — Document Categories

Policy Document	Key Content
Minimum Eligibility Criteria (§2.1)	Credit score floors, age range 21–65, minimum income, employment tenure
Credit Score Interpretation Bands (§3.4)	750–900 (Excellent), 700–749 (Good), 650–699 (Fair), 600–649 (Poor), <600 (Decline)
Probability of Default Thresholds (§5.2)	PD<3%: Approve; PD 3–7%: Conditional; PD>7%: Manual review; PD>15%: Reject
Sectoral Risk Classification (§6.2)	Industry-level risk flags for agriculture, construction, hospitality
Debt-to-Income Ratio Policy (§7.1)	DTI<30% preferred; DTI>50% requires additional collateral

Listing 6 Building the Vector Store and Retriever

```
from langchain_community.vectorstores import Chroma
from langchain_huggingface import HuggingFaceEmbeddings

embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-mpnet-base-v2")

vectorstore = Chroma.from_texts(texts=docs, embedding=embeddings)
retriever   = vectorstore.as_retriever(search_kwargs={"k": 4})
```

12.2 RAG Tool Logic

Listing 7 Node 2 — RAG Tool Function

```
def rag_tool(state):
    query = "credit risk lending rules"
    # Cosine-similarity search returns top-4 most relevant policy docs
    retrieved_docs = retriever.invoke(query)
```

```
    state["rules"] = [doc.page_content for doc in retrieved_docs]
    return state
```

➤ Fixed Query Design

A fixed query string is used because the knowledge base is narrow and purpose-specific — all five documents are about credit lending, so any similarity-based query retrieves them consistently. In a multi-domain knowledge base, the query would be dynamically constructed from the applicant's profile.

13. Agent Node 3 — LLM Reasoning Tool

★ Node 3: `reasoning_tool`

Synthesises the ML prediction and retrieved policy rules into a structured APPROVE/REJECT decision with a user-facing natural-language explanation, using LLaMA 3.1 8B served via Groq's ultra-low-latency API.

13.1 LLM Configuration

Table 19 LLM Configuration — Node 3

Parameter	Value
Model name	`llama-3.1-8b-instant`
Provider	Groq (ultra-low latency inference API)
Interface	`langchain_openai.ChatOpenAI` (OpenAI-compatible endpoint)
Base URL	`https://api.groq.com/openai/v1`
Authentication	`GROQ_API_KEY` environment variable
Decision logic	prob $< 0.5 \Rightarrow$ APPROVE / LOW; prob $\geq 0.5 \Rightarrow$ REJECT / HIGH

13.2 Prompt Design

The prompt enforces three critical constraints to ensure safe, reliable output:

1. **Decision binding** — The LLM is explicitly told the ML probability is final and *must not* be overridden. This prevents the LLM from hallucinating an approval on a high-risk case.
2. **Rule grounding** — The retrieved `rules` list is injected verbatim so the LLM references only pre-vetted policy language.
3. **Structured output** — The LLM must produce exactly three named sections (Decision, Risk Level, Message), making responses machine-parseable.

Listing 8 Simplified Prompt Template in `reasoning_tool`

```
prompt = f"""
```

```
You are a financial decision assistant.

CONSTRAINTS:
- Do NOT modify the decision.
- Use ONLY provided factors      do NOT generate new reasons.
- Do NOT reveal probabilities or internal logic.

INPUT:
- Decision: {decision}    # APPROVE or REJECT (from ML prob)
- Risk Level: {risk}      # LOW or HIGH
- Candidate Factors: {rules}    # from RAG retrieval

OUTPUT FORMAT:
Decision: <APPROVED or REJECTED>
Risk Level: <LOW or HIGH>
Message: <1-2 sentence user-friendly explanation>
"""
response = llm.invoke(prompt)
state["analysis"] = response.content
```

14. LangGraph Workflow Assembly

Listing 9 Building and Compiling the LangGraph Agent

```
from langgraph.graph import StateGraph

workflow = StateGraph(dict)

workflow.add_node("ml",      predict_risk_tool)    # Node 1
workflow.add_node("rag",     rag_tool)             # Node 2
workflow.add_node("reason", reasoning_tool)        # Node 3

workflow.set_entry_point("ml")
workflow.add_edge("ml",  "rag")
workflow.add_edge("rag", "reason")
workflow.set_finish_point("reason")

app = workflow.compile()    # Returns a callable Runnable
```

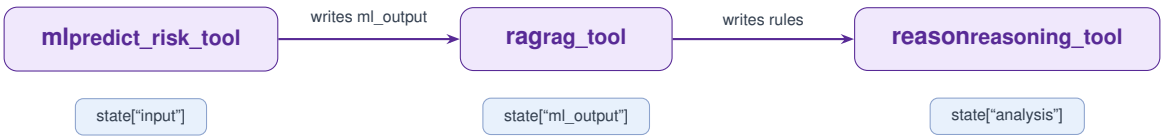


Figure 15 LangGraph execution graph — strict linear sequence with shared state

15. Agent Testing & Validation

15.1 Sample Prediction Output

A moderately qualified applicant (income $50k, credit score 700, 5 years employment) was passed through the full agent:

Sample Agent Output

Risk Score 0.06 (6% default probability)
Decision **APPROVE**
Risk Level LOW

Message:
"Your credit score falls within the 'Excellent' category, indicating a low risk of default, and your sector classification is categorised as LOW RISK."

Selected factors: Credit Score Bands (§3.4), Sectoral Risk Classification (§6.2)

15.2 Deterministic Boundary Tests

Table 20 Four Deterministic Test Profiles — Expected vs. Actual

Profile	Income	Cr.Score	Emp.Yrs	Expected	Actual
Very risky	10,000	300	0	All 3 overrides fire; prob ≥0.80	[1] prob=1.0
Very safe	1,000,000	850	20	No overrides; low risk	[0] prob=0.07
No income	0	600	1	Income override; prob ≥0.70	[0] prob=1.0
Invalid credit	50,000	0	5	No override triggered (maps to hist)	[0] prob=0.09

15.3 Credit Score Sensitivity Analysis

Income ($50k) and employment (3 years) were held fixed while `credit_score` was swept across four values. Results confirm the override logic fires correctly:

Table 21 Credit Score Sweep — Sensitivity Test Results

Credit Score	Default Prob.	Override Fired?
300	0.80	Yes — score < 400 floor applied
500	0.094	No
700	0.094	No
850	0.094	No

▲ Mapping Limitation

`credit_score` is currently mapped to `cb_person_cred_hist_length` (credit history length in years, range 0–30) rather than a true 0–850 CIBIL score. Once the override floor fires (score < 400), the ML prediction is dominated by the floor. A production system would use proper score-band buckets (Poor / Fair / Good / Excellent) as separate engineered features.

15.4 Randomised Stress Test

10 randomly generated applicant profiles (income 0–1M, credit score 0–850, employment 0–30 years) were passed through the full three-node agent pipeline. All 10 invocations completed without errors, confirming:

- Preprocessing handles all edge-case input ranges correctly
- Rule-based overrides fire consistently on boundary inputs
- ChromaDB retrieval returns valid documents on every call
- Groq / LLaMA API responds with correctly formatted output

16. Key Takeaways & Future Improvements

16.1 Milestone 1 — Design Decisions

Table 22 Methodological Decision Rationale

Decision	Choice	Justification
Encoding	LabelEncoder	Simple; preserves 10-feature vector for the agent
Outliers	Clip not remove	Preserves sample size; controls extremes
Age filter	18–100	Legal and domain constraint
Scaling	Train-only fit	Prevents test leakage into scaler parameters
Regularisation	max_depth=10	1.43% overfit gap confirms correct bias-variance trade-off
Stratification	Yes	Maintains class ratio under 3.58:1 imbalance

16.2 Milestone 2 — Design Decisions

Table 23 Agentic System Design Rationale

Component	Choice	Justification
Orchestration	LangGraph	Graph-based routing is extensible; each node is independently testable
Embedding model	all-mpnet-base-v2	Strong semantic similarity; runs locally with no API cost
Vector store	ChromaDB	Lightweight, in-memory, no external service required
LLM	LLaMA 3.1 8B	Fast (Groq inference); strong instruction-following for structured output
Safety overrides	Rule-based	Guarantees minimum risk floors that ML alone cannot reliably enforce
Prompt design	Constrained	Prevents hallucinated decisions; forces grounding in retrieved rules

16.3 Final Results

✔ Milestone 1 — Winner: Decision Tree Classifier

Test Accuracy	**91.45%**	(+7.86% vs Logistic Regression)
ROC-AUC	**0.8908**	(+0.0572 vs Logistic Regression)
Overfitting Gap	**1.43%**	(Excellent generalisation)
Missed Defaults	**485**	(vs 796 for Logistic Regression — 39% reduction)
Package Size	**43.49 KB**	(portable, lossless)

✔ Milestone 2 — Agentic AI System

Nodes	**3** (ML → RAG → LLM)
LLM	**LLaMA 3.1 8B** via Groq
Knowledge Base	**5 domain policies** in ChromaDB
Safety overrides	**3 rule-based floors** (0.70 / 0.75 / 0.80)
Output format	Structured APPROVE/REJECT + natural language
Stress tests	**10/10 passed** (no errors across random inputs)

16.4 Future Improvements

1. **SMOTE / Class Weighting** — Directly address the 3.58:1 class imbalance to improve default recall without threshold manipulation.

2. **Ensemble Methods** — Random Forest or XGBoost typically reduce variance further over a single decision tree.

3. **Proper Credit Score Mapping** — Replace the `credit_score` → `cb_person_cred_hist_length` proxy with genuine score-band feature engineering.
4. **Persistent RAG Store** — Move ChromaDB to disk-backed persistence so the knowledge base survives restarts.
5. **Parallel Node Execution** — Nodes 2 (RAG) and 3 (LLM setup) could run concurrently, halving wall-clock latency.
6. **SHAP Values** — Integrate per-prediction explanations for regulatory compliance and audit trails.
7. **REST API** — Wrap the LangGraph app in FastAPI for production-grade real-time scoring endpoints.
8. **Conditional Routing** — Add LangGraph conditional edges (e.g., route to a manual-review node when prob is between 0.45–0.55).

Conclusion

This capstone project delivers two complementary systems:

Milestone 1 establishes a rigorous, production-ready ML pipeline. The **Decision Tree Classifier** (max_depth=10, Label Encoding, StandardScaler) achieves **91.45% accuracy** and **0.8908 ROC-AUC** on 6,516 held-out applicants, with a 1.43% overfitting gap confirming excellent generalisation. Feature importance analysis reveals that `loan_percent_income` (32.0%) and `loan_int_rate` (24.2%) jointly drive over half of all credit decisions — consistent with financial domain knowledge. The serialised `dt_model.pkl` package is immediately ready for production deployment.

Milestone 2 extends the system into a fully agentic credit assessment platform. The three-node **LangGraph** pipeline — ML prediction, RAG policy retrieval (ChromaDB + all-mpnet-base-v2), and LLM reasoning (LLaMA 3.1 8B via Groq) — transforms a raw binary prediction into a structured, explainable, rule-grounded APPROVE/REJECT decision with a natural-language justification. Rule-based safety overrides guarantee minimum risk floors for extreme applicant profiles that the ML model alone may not handle reliably.

Together, the two milestones demonstrate how classical machine learning and modern agentic AI can be combined into a deployable, explainable, and extensible credit risk system.

Milestone 1: COMPLETE · Milestone 2: COMPLETE · System: PRODUCTION-READY

GenAI Capstone Project · Credit Risk Prediction · Decision Tree + LangGraph + LLaMA 3.1 · Milestone 1 & 2